Batch Reinforcement Learning: Model-Based MLE

Alexander Burns - A06690908

AA 228: Decision Making Under Uncertainty · Project 2

November 1st 2026

Abstract

Batch reinforcement learning (RL) is an offline method that trains an agent to maximize its expected total reward based on sets of state, action, and reward observations. An algorithm is proposed to obtain an optimal policy of the estimated Markov decision process (MDP) for three different RL problems with number of states and actions scaling over $(n_s, n_a) = (100, 4), (50000, 7), (302020, 9)$. The proposed model-based algorithm combines maximum likelihood estimation (MLE) to reformat the problem as a MDP and utilizes value iteration to solve it. Additionally, to avoid the infeasible data storage size associated with the transition matrix in MLE $O(|S|^2|A|)$ for larger datasets, sparse storage is implemented, reducing the complexity to $O(X)$, where X is the number of observations recorded. The algorithm proposed obtains a policy in run times of $t = 0.01, 4.00, 28.75$ seconds, respectively.

1 Algorithm Overview

Model-based MLE is chosen due to the inherent non-exploitative offline nature of batch RL. As a result, the MLE of the transition and reward functions can be computed for the entire observation dataset efficiently. However, one flaw is that neighboring states are treated as unrelated preventing the structure of the problem from being exploited, a limitation shared by model-free approaches like Q-learning. Model-free approaches were not considered since they handle one sample at a time and add complexity through hyperparameters, necessitating the need for tuning and refinement.

1.1 Improved MLE Algorithm

Improvements made to the MLE algorithm defined in Chapter 16 of "Algorithms for Decision Making" [1] include vectorizing the computation of the count and reward sums, sparse transition matrix storage, and forcing the convergence of undiscounted problems.

Vectorizing Count and Reward Computation. The MLE algorithm detailed as Algorithms 16.1 & 16.2 [1] assumes online RL and is therefore inefficient for batch RL problems where the observation dataset is already available. Instead of looping over one observation at a time in Algorithm 16.1 and then subsequently looping over every state action combination in Algorithm 16.2, the computation of the counts $N(s, a, s')$ and rewards $\rho(s, a)$ is instead vectorized. This efficiency improvmenets is particularly evident at larger dataset sizes.

Sparse Transition Matrix Storage. A compressed sparse row (CSR) matrix is instead used to sparsely store the full $|S|^2|A|$ transition matrix required to explain the relationship between the current state, the selected action, and the resulting state. This greatly reduces the transition matrix's occupied memory, which otherwise would exceed available memory for the medium & large

datasets. The CSR matrix instead only stores the non-zero entries and their associated positions within the larger data structure, meaning that memory scales only with the number of recorded observations instead of quadratic over S.

Forced Convergence of Undiscounted Problems To address potential issues with convergence of undiscounted problems ($\gamma = 1.00$), an artificial discount factor was applied. The chosen discount factor was $\gamma = 0.99$ to encourage convergence in reasonable time. For the medium dataset, this introduces large modeling errors as the number of steps taken increases, since future rewards are eventually heavily discounted instead of being weighed as equal.

1.2 Unobserved Actions

For states with only some of the possible actions observed, the MLE estimates of the transition and reward functions are undefined for the unobserved actions. To account for this, the action value function corresponding to the absent actions is set to $Q(s, a) = -\infty$ which creates zero impact to the utility of a state and forces the resulting policy to only select actions observed. This restriction on the set of possible actions the policy can take at particular states prevents the policy from exploring potentially advantageous unobserved actions, limiting the obtained solution.

1.3 Terminal States

The medium batch RL problem contains terminal states, which exist once the mountain car has reached its goal, the flag. Since the position of the flag varies and is not provided to the agent, the terminal position is unknown and therefore must be approximated by the largest position observed in a state. As a result, every new state that the agent encounters that occurs at or after the maximum observed position in the training dataset is treated as terminal, with an associated utility of zero ($U(s) = 0$) since no further rewards can be yielded.

1.4 States with No Actions Observed

Some states in the data set only occur as the subsequent state s' and therefore have no observed actions for them. Two possible approaches to account for this are: 1. Estimate the problem's underlying structure, or 2. Estimate what the utility might be. While possible inferences could be made about each dataset, I instead chose a conservative approach and assumed that the utility of the unobserved state is simply the average of the utilities of all other states with observed actions.

2 Results

Table 1: Key metrics of the batch RL results

Dataset	States	Actions	Raw Score	Runtime (s)
Small	100	4	35.52	0.01
Medium	50000	7	54.38	4.00
Large	302020	9	545.44	28.75

AI Policy

AI (Claude) was used to assist with learning the tradeoffs between model-based and model-free approaches to RL. I also asked Claude to explain how to handle offline RL in comparison to online RL. Claude was also used to suggest alternatives to Matlab and Julia commands that I am more accustomed to, as well as explain the syntax of commands I implemented. AI was not used to write code directly or structure the implementation of my algorithm. Minimal AI usage was used to learn general formatting for LaTeX.

References

[1] Mykel J. Kochenderfer, Tim A. Wheeler, and Kyle H. Wray. *Algorithms for Decision Making*. MIT Press, 2022. URL `https://algorithmsbook.com/files/dm.pdf`.

Appendix

Code 1: Batch RL MLE Sparse (Python)

```python
# === Alexander Burns | AA 228 Decision Making Under Uncertainty | Project 2 | Batch Reinforcement Learning ===
"""
Model-Based Maximum Likelihood Estimation with Value Iteration + Sparse Storage
--> Uses Value Iteration to apply Bellman backup until the utility is ~ constant
--> Uses Sparse Storage to scale the model-based approach to medium/large datasets (avoid storage of |S|x|A|x|S| which is sometimes infeasible)
"""

# === Imports ===
import sys
import time

import numpy as np
import pandas as pd
from scipy.sparse import csr_matrix


# === General Helper Functions ===
def read_csv(filename):
    """Read the CSV data file and then return columns as numpy arrays"""
    data = pd.read_csv(filename, sep=",", header=0)
    D_numpy = data.to_numpy()

    # = Extract Columns of Data =
    s = D_numpy[:, 0] - 1  # index shift
    a = D_numpy[:, 1] - 1  # index shift
    r = D_numpy[:, 2]
    sp = D_numpy[:, 3] - 1  # index shift

    return s, a, r, sp


```

```python
def save_results(outputfilename, policy_pi, dt):
    """Write the results of the batch RL to disk"""

    # == Write the policy to disk ==
    np.savetxt(outputfilename, policy_pi, fmt="%d")

    # == Write the time it took for the algorithm to finish ==
    print(f"Total Execution Time: {dt} s")
    np.savetxt(outputfilename + ".time", np.array([dt]), fmt="%.6f")


# === Math Helper Functions ===
def convert_to_MDP(s, a, r, sp, n_states, n_actions):
    """Based on Ch 16 Algorithm 16.1 & 16.2
    -> Vectorizes the additon of 1 to N[s,a,s'] and r to rho[s,a]
       -> Only for sparse storage of N[s,a,s'] to avoid memory issues of |S|x|A|x|S|
    -> Skips update!/other functions present in 16.1 since this is batched RL and no new observations are observed
    -> Converts the maximum likelihood model to an MDP
       -> Returns ApproximateTransition and Reward Function"""

    # === Algorithm 16.1: Update Transition Counts/Reward Sum ===

    # Combine States/Actions into 1D array
    sa_array = s * n_actions + a

    # Count number and reward corresponding to state action pair
    N_sa = np.bincount(sa_array, minlength=n_states * n_actions)
    rho = np.bincount(sa_array, weights=r, minlength=n_states * n_actions)

    # === Algorithm 16.2: Compute Approximate Transition and Reward Functions ===

    # = Create the sparse T(sp|s,a) through a compressed sparse row matrix =
    # NOTE: EQ 16.1: T(sp|s,a) = N(s,a,sp) / N(s,a)
    N_sasp = csr_matrix(
        (np.ones(len(s)), (sa_array, sp)), shape=(n_states * n_actions, n_states)
    )
    T = csr_matrix(N_sasp.multiply(1 / np.maximum(N_sa, 1)[:, None]))

    # = Create the MLE of the reward function as the mean reward =
    # NOTE: EQ 16.2: R(s,a) = rho(s,a) / N(s,a)
    R = rho / np.maximum(N_sa, 1)

    return N_sa, T, R


def value_iteration(N_sa, T, R, s, discount_factor, n_states, n_actions):
    """Alternative to Ch 16 Algorithm 16.3 -> Algorithm 7.8: Using value iteration over linear programming
    as mentioned in section 16.2.1. Modifications include:
    -> Accounting for terminal states for the medium dataset (utility is zero)
    -> Accounting for states with no actions (utility is average utility)"""

    # === Algorithm 7.8: Iterate Values of U(s) ===
    err = np.inf
    res = 1e-6
    U = np.zeros(n_states)

    while err > res:
```

```
        # EQ 7.16 Bellman Backup Equation
        Q = R + discount_factor * (T @ U)
        Q = np.where(N_sa > 0, Q, -np.inf).reshape(n_states, n_actions)

        # Evaluate U as max of Q(s,a) over actions a
        U_n = np.max(Q, axis=1)

        # Account for Terminal States in medium
        if n_states == 50000:
            # NOTE: From problem statement, 1+pos+500*vel is the state integer
            # -> therefore, s = pos + 500 * vel -> max position achieved can be found
            max_pos = np.max(s % 500) + 1
            term_states = max_pos <= np.arange(n_states) % 500

            # Set U(term) = 0 since future states past term do not have any reward
            U_n[term_states] = 0

        # Account for states with no actions (mean reward)
        U_n[np.isinf(U_n)] = U_n[~np.isinf(U_n)].mean()  # as

        # Check Convergence
        err = np.max(np.abs(U_n - U))
        U = U_n

    # = Recompute Q after U convergence =
    Q = R + discount_factor * (T @ U)
    Q = np.where(N_sa > 0, Q, -np.inf).reshape(n_states, n_actions)

    return Q


# === Main Functions ===
def RL_algorithm(s, a, r, sp, discount_factor, n_states, n_actions):

    # = Convert MLE to MDP =
    N_sa, T, R = convert_to_MDP(s, a, r, sp, n_states, n_actions)

    # = Iterate U to Convergence, Return Q =
    Q = value_iteration(N_sa, T, R, s, discount_factor, n_states, n_actions)

    # = Extract Policy From Q =
    pi_s = np.argmax(Q, axis=1) + 1  # EQ 7.11 Policy Extraction (with index shift back)

    return pi_s


def main():
    # === Initialize ===
    if len(sys.argv) != 3:
        raise Exception("usage: python project1.py <infile>.csv <outfile>.policy")

    inputfilename = sys.argv[1]
    outputfilename = sys.argv[2]

    # = Assign discount factor & # of states/actions based on data =
    if "small" in inputfilename:
        discount_factor = 0.95
        n_states = 100
        n_actions = 4
```

```
    elif "medium" in inputfilename:
        discount_factor = 0.99
        n_states = 50000
        n_actions = 7
    elif "large" in inputfilename:
        discount_factor = 0.95
        n_states = 302020
        n_actions = 9
    else:
        print("Error declaring csv file")

    # = Read CSV data file =
    s, a, r, sp = read_csv(inputfilename)

    # === Execute Algorithm ===
    tic = time.perf_counter()
    policy_pi = RL_algorithm(s, a, r, sp, discount_factor, n_states, n_actions)
    toc = time.perf_counter()

    # === Write Results ===
    save_results(outputfilename, policy_pi, toc - tic)


if __name__ == "__main__":
    main()
```